

O Sistema de Tipos

Tipos

Apresentamos agora um sistema de tipos simples para a nossa linguagem funcional.

- **Sintaxe abstracta**

$$\begin{aligned} \langle type \rangle ::= & \mathbf{Int} \mid \mathbf{Bool} \mid \langle type \rangle \rightarrow \langle type \rangle \\ & \mid \mathbf{Prod}(\langle type \rangle, \dots, \langle type \rangle) \\ & \mid \mathbf{Sum}(\langle type \rangle, \dots, \langle type \rangle) \\ & \mid \mathbf{List} \langle type \rangle \end{aligned}$$

- **Abreviaturas**

$$\begin{aligned} \langle type \rangle \times \dots \times \langle type \rangle & \doteq \mathbf{Prod}(\langle type \rangle, \dots, \langle type \rangle) \\ \mathbf{Unit} & \doteq \mathbf{Prod}() \\ \langle type \rangle + \dots + \langle type \rangle & \doteq \mathbf{Sum}(\langle type \rangle, \dots, \langle type \rangle) \end{aligned}$$

- **Precedências:** **List**, $\times$, $+$, $\rightarrow$

- **Associatividade:** $\times$, $+$ à esquerda; $\rightarrow$ à direita

Contextos

Para definir a relação de tipificação entre termos e tipos precisamos de introduzir a noção de *contexto* para declarar o tipo das variáveis (e dos padrões).

- Um **contexto** é uma lista de associações de tipos a padrões

$$\langle context \rangle ::= \mid \langle context \rangle, \langle pat \rangle : \langle type \rangle$$

com a restrição de uma variável não poder ocorrer mais do que uma vez num contexto.

- Um **juízo de tipo** tem a forma

$$\langle context \rangle \vdash \langle exp \rangle : \langle type \rangle$$

Sistema de tipos

- Definimos agora um sistema de tipos com base num conjunto de regras de inferência de tipos que especificam os juízos de tipos válidos.

- Usaremos as seguintes **meta-variáveis**

Γ	$\langle context \rangle$	x	$\langle var \rangle$
θ, τ, σ	$\langle type \rangle$	p	$\langle pat \rangle$
e	$\langle exp \rangle$	k, n	$\mathbb{N}$

Regras de inferência de tipos

$$\frac{}{\Gamma, x : \tau \vdash x : \tau}$$

$$\frac{\Gamma, p : \sigma \vdash e : \tau}{\Gamma \vdash \lambda p. e : \sigma \rightarrow \tau}$$

$$\frac{\Gamma \vdash e_1 : \sigma \rightarrow \tau \quad \Gamma \vdash e_2 : \sigma}{\Gamma \vdash e_1 e_2 : \tau}$$

Regras de inferência de tipos

$$\frac{}{\Gamma \vdash n : \mathbf{Int}} \text{ para cada } n \in \mathbf{N}$$

$$\frac{\Gamma \vdash e : \mathbf{Int}}{\Gamma \vdash \neg e : \mathbf{Int}}$$

$$\frac{\Gamma \vdash e_1 : \mathbf{Int} \quad \Gamma \vdash e_2 : \mathbf{Int}}{\Gamma \vdash e_1 \mathbf{bop} e_2 : \mathbf{Int}} \text{ para } \mathbf{bop} \in \{+, -, *, \text{div}, \text{mod}\}$$

Regras de inferência de tipos

$$\frac{}{\Gamma \vdash \mathbf{True} : \mathbf{Bool}}$$

$$\frac{}{\Gamma \vdash \mathbf{False} : \mathbf{Bool}}$$

$$\frac{\Gamma \vdash e : \mathbf{Bool}}{\Gamma \vdash \neg e : \mathbf{Bool}}$$

$$\frac{\Gamma \vdash e_1 : \mathbf{Bool} \quad \Gamma \vdash e_2 : \mathbf{Bool}}{\Gamma \vdash e_1 \mathbf{bop} e_2 : \mathbf{Bool}} \text{ para } \mathbf{bop} \in \{\vee, \wedge\}$$

Regras de inferência de tipos

$$\frac{\Gamma \vdash e_1 : \mathbf{Int} \quad \Gamma \vdash e_2 : \mathbf{Int}}{\Gamma \vdash e_1 \mathbf{bop} e_2 : \mathbf{Bool}} \text{ para } \mathbf{bop} \in \{=, \neq, <, >, \leq, \geq\}$$

$$\frac{\Gamma \vdash e_1 : \mathbf{Bool} \quad \Gamma \vdash e_2 : \theta \quad \Gamma \vdash e_3 : \theta}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \theta}$$

Regras de inferência de tipos

$$\frac{}{\Gamma \vdash \langle \rangle : \mathbf{Unit}}$$

$$\frac{\Gamma \vdash e_1 : \theta_1 \ \dots \ \Gamma \vdash e_n : \theta_n}{\Gamma \vdash \langle e_1, \dots, e_n \rangle : \theta_1 \times \dots \times \theta_n}$$

$$\frac{\Gamma \vdash e : \theta_1 \times \dots \times \theta_n}{\Gamma \vdash e.k : \theta_k} \text{ para } k \in \{1, \dots, n\}$$

Regras de inferência de tipos

$$\frac{\Gamma \vdash e : \theta_k}{\Gamma \vdash @k e : \theta_1 + \dots + \theta_n} \text{ para } k \in \{1, \dots, n\}$$

$$\frac{\Gamma \vdash e : \theta_1 + \dots + \theta_n \quad \Gamma \vdash e_1 : \theta_1 \rightarrow \theta \ \dots \ \Gamma \vdash e_n : \theta_n \rightarrow \theta}{\Gamma \vdash \text{sumcase } e \text{ of } (e_1, \dots, e_n) : \theta}$$

Regras de inferência de tipos

$$\frac{}{\Gamma \vdash \text{nil} : \mathbf{List } \theta} \quad \frac{\Gamma \vdash e_1 : \theta \quad \Gamma \vdash e_2 : \mathbf{List } \theta}{\Gamma \vdash (e_1 :: e_2) : \mathbf{List } \theta}$$

$$\frac{\Gamma \vdash e : \mathbf{List } \theta \quad \Gamma \vdash e_1 : \theta' \quad \Gamma \vdash e_2 : \theta \rightarrow \mathbf{List } \theta \rightarrow \theta'}{\Gamma \vdash \text{listcase } e \text{ of } (e_1, e_2) : \theta'}$$

Regras de inferência de tipos

$$\frac{\Gamma, p_1 : \theta_1, \dots, p_n : \theta_n \vdash e : \theta}{\Gamma, \langle p_1, \dots, p_n \rangle : \theta_1 \times \dots \times \theta_n \vdash e : \theta}$$

$$\frac{\Gamma \vdash e_1 : \theta_1 \ \dots \ \Gamma \vdash e_n : \theta_n \quad \Gamma, p_1 : \theta_1, \dots, p_n : \theta_n \vdash e : \theta}{\Gamma \vdash \text{let } p_1 \equiv e_1, \dots, p_n \equiv e_n \text{ in } e : \theta}$$

Regras de inferência de tipos

$$\frac{\Gamma' \vdash e_1 : \theta_1 \quad \dots \quad \Gamma' \vdash e_n : \theta_n \quad \Gamma' \vdash e : \theta}{\Gamma \vdash \text{letrec } p_1 \equiv e_1, \dots, p_n \equiv e_n \text{ in } e : \theta} \text{ com } \Gamma' = \Gamma, p_1 : \theta_1, \dots, p_n : \theta_n$$

Exercícios

Considere a seguinte expressão FACT

$\text{letrec fact} \equiv \lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * \text{fact } (n - 1) \text{ in fact}$

Construa uma árvore de prova do juízo

$\vdash \text{FACT} : \mathbf{Int} \rightarrow \mathbf{Int}$

Exercícios

Considere a seguinte expressão APP

$\text{letrec append} \equiv \lambda x. \lambda y. \text{listcase } x \text{ of } (y, \lambda h. \lambda t. h :: \text{append } t y) \\ \text{in append } (1 :: \text{nil}) (2 :: 7 :: 8 :: \text{nil})$

Construa uma árvore de prova do juízo

$\vdash \text{APP} : \mathbf{List Int}$

Exercícios

Construa uma extensão da linguagem de programação funcional, por forma a incluir um tipo de árvores binárias (com números inteiros nos nós).

Defina sintaxe abstracta, regras de inferência de tipos, e regras de avaliação CBV apropriadas.

As árvores binárias podem ser vistas como estruturas de dados construídas à custa das alternativas e dos tuplos, vendo os seus construtores e eliminadores como açúcar sintático.

Apresente esta definição alternativa.

FUNcbn

Uma Linguagem Funcional “call-by-name”

Uma linguagem funcional “call-by-name”

- Veremos agora uma pequena linguagem funcional com uma semântica que segue a ordem normal de avaliação. Ou seja, uma semântica “call-by-name” (CBN).
- A **sintaxe** da linguagem é idêntica à que vimos para a linguagem CBV, excepto para o caso das definições recursivas.

- ▶ A semântica CBN permite um **operador de ponto fixo**

$\langle exp \rangle ::= \text{rec } \langle exp \rangle$

- A semântica da linguagem CBN é bastante diferente da CBV: a avaliação dos operandos das aplicações, dos componentes dos tuplos e das alternativas é adiada até que seja evidente que esses valores são necessários para determinar o resultado do programa.

Uma linguagem funcional “call-by-name”

- O **sistema de tipos** para esta linguagem CBN é idêntico ao que vimos para a linguagem CBV. Apenas se acrescenta a seguinte regra de inferência de tipos

$$\frac{\Gamma \vdash e : \theta \rightarrow \theta}{\Gamma \vdash \text{rec } e : \theta}$$

- Há expressões cuja avaliação CBN termina, embora possam divergir com uma avaliação CBV.
- Será possível definir **estruturas de dados infinitas** (como por exemplo as “lazy lists”).

Formas canónicas

- A linguagem usa uma semântica de avaliação “call-by-name”.
- $\Rightarrow$ relaciona as expressões com as formas canónicas da linguagem

$\langle exp \rangle \Rightarrow \langle cfm \rangle$

- Teremos **formas canónicas de diferentes tipos**

$\langle cfm \rangle ::=$

- $\lambda \langle var \rangle . \langle exp \rangle$
- $\dots \mid -2 \mid -1 \mid 0 \mid 1 \mid 2 \mid \dots$
- $\text{True} \mid \text{False}$
- $\langle \langle exp \rangle, \dots, \langle exp \rangle \rangle$
- $@ \langle tag \rangle \langle exp \rangle$
- $\text{nil} \mid \langle exp \rangle :: \langle exp \rangle$

Semântica de avaliação “call-by-name”

- Formas canónicas

$$\frac{}{z \Rightarrow z}$$

- Aplicação

$$\frac{e \Rightarrow \lambda v. \hat{e} \quad \hat{e}[e'/v] \Rightarrow z}{e e' \Rightarrow z}$$

Semântica de avaliação “call-by-name”

- Operadores unários

$$\frac{e \Rightarrow [i]}{-e \Rightarrow [-i]}$$

- Operadores binários

$$\frac{e_1 \Rightarrow [i_1] \quad e_2 \Rightarrow [i_2]}{e_1 \text{ bop } e_2 \Rightarrow [i_1 \text{ bop } i_2]} \text{ para } \text{bop} \in \{+, -, *, =, \neq, <, >, \leq, \geq\}$$

$$\frac{e_1 \Rightarrow [i_1] \quad e_2 \Rightarrow [i_2]}{e_1 \text{ bop } e_2 \Rightarrow [i_1 \text{ bop } i_2]} \text{ para } \text{bop} \in \{\text{div}, \text{mod}\} \text{ e } i_2 \neq 0$$

Açúcar sintáctico

- Operadores booleanos

$$\neg e \doteq \text{if } e \text{ then False else True}$$

$$e \vee e' \doteq \text{if } e \text{ then True else } e'$$

$$e \wedge e' \doteq \text{if } e \text{ then } e' \text{ else False}$$

Desta forma a avaliação de $\vee$ e $\wedge$ é em “curto-circuito”.

Semântica de avaliação “call-by-name”

- Expressões condicionais

$$\frac{e_1 \Rightarrow \text{True} \quad e_2 \Rightarrow z}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow z}$$

$$\frac{e_1 \Rightarrow \text{False} \quad e_3 \Rightarrow z}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow z}$$

Semântica de avaliação “call-by-name”

• Tuplos

$$\frac{e \Rightarrow \langle e_1, \dots, e_n \rangle \quad e_k \Rightarrow z}{e.k \Rightarrow z} \text{ para } k \in \{1, \dots, n\}$$

Semântica de avaliação “call-by-name”

• Alternativas

$$\frac{e \Rightarrow @k e' \quad e_k e' \Rightarrow z}{\text{sumcase } e \text{ of } (e_1, \dots, e_n) \Rightarrow z} \text{ para } k \in \{1, \dots, n\}$$

Semântica de avaliação “call-by-name”

• Listas

$$\frac{e \Rightarrow \text{nil} \quad e_e \Rightarrow z}{\text{listcase } e \text{ of } (e_e, e_{ne}) \Rightarrow z}$$

$$\frac{e \Rightarrow e' :: e'' \quad e_{ne} e' e'' \Rightarrow z}{\text{listcase } e \text{ of } (e_e, e_{ne}) \Rightarrow z}$$

Semântica de avaliação “call-by-name”

• Recursividade

$$\frac{e (\text{rec } e) \Rightarrow z}{\text{rec } e \Rightarrow z}$$

Açúcar sintático

- Padrões, definições “let” e listas podem ser definidas como açúcar sintático do mesmo modo que na linguagem CBV.
- O operador de ponto fixo pode ser usado para definições “letrec” mais gerais do que tínhamos na linguagem CBV

$\langle exp \rangle ::= \text{letrec } \langle pat \rangle \equiv \langle exp \rangle, \dots, \langle pat \rangle \equiv \langle exp \rangle \text{ in } \langle exp \rangle$

- Onde

$$\begin{aligned} \text{FV}(\text{letrec } p_1 \equiv e_1, \dots, p_n \equiv e_n \text{ in } e) \\ = (\text{FV}(e_1) \cup \dots \cup \text{FV}(e_n) \cup \text{FV}(e)) - (\text{FV}(p_1) \cup \dots \cup \text{FV}(p_n)) \end{aligned}$$

Açúcar sintático

- Letrec

$\text{letrec } p_1 \equiv e_1 \text{ in } e \doteq \text{let } p_1 \equiv \text{rec } (\lambda p_1. e_1) \text{ in } e$

$\begin{aligned} \text{letrec } p_1 \equiv e_1, \dots, p_n \equiv e_n \text{ in } e \\ \doteq \text{let } \langle p_1, \dots, p_n \rangle \equiv \text{rec } (\lambda \langle p_1, \dots, p_n \rangle. \langle e_1, \dots, e_n \rangle) \text{ in } e \end{aligned}$

Exemplo

Considere a seguinte função que testa se duas listas são iguais

$\text{eqlist} : \text{List Int} \rightarrow \text{List Int} \rightarrow \text{Bool}$

e compare a avaliação CBN e CBV do seguinte programa

```
letrec eqlist  $\equiv \lambda l_1. \lambda l_2.$ 
  listcase  $l_1$  of (
    listcase  $l_2$  of (True,  $\lambda h_2. \lambda t_2.$  False),
     $\lambda h_1. \lambda t_1.$  listcase  $l_2$  of (False,  $\lambda h_2. \lambda t_2.$   $h_1 = h_2 \wedge \text{eqlist } t_1 t_2$ )
  )
in eqlist (1 :: 2 :: 3 :: nil) (3 :: 2 :: 1 :: nil)
```

Como se comporta a função eqlist com listas infinitas?

Estruturas de dados infinitas

A semântica CBN permite definir estruturas de dados infinitas, como por exemplo as seguintes “lazy lists”

A lista infinita de zeros

$\text{letrec zerolist} \equiv 0 :: \text{zerolist} \dots$

ou simplesmente, $\text{rec } (\lambda l. 0 :: l)$

A lista infinita de números naturais

$\begin{aligned} \text{letrec map} &\equiv \lambda f. \lambda l. \text{listcase } l \text{ of } (\text{nil}, \lambda h. \lambda t. f h :: \text{map } f t) \\ \text{in } &\text{rec } (\lambda l. 0 :: \text{map } (\lambda x. x + 1) l) \end{aligned}$

Apresente uma definição alternativa para a lista de naturais chamada natlist.